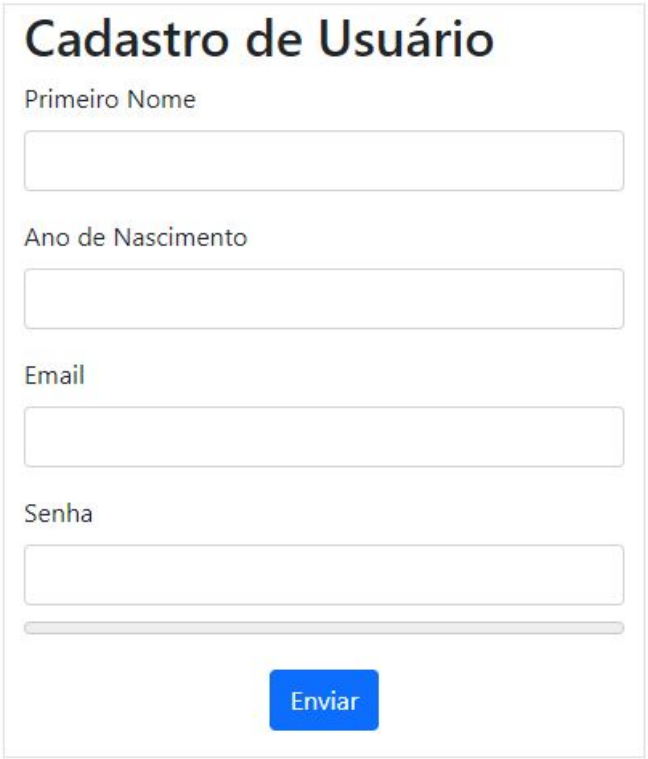
# Aula 08 - Teste Funcional Combinatorial - com Selenium

**Profa**. Dra. Lina Garcés

# DESAFIO!

# Exemplo - HU: Cadastro de Usuário

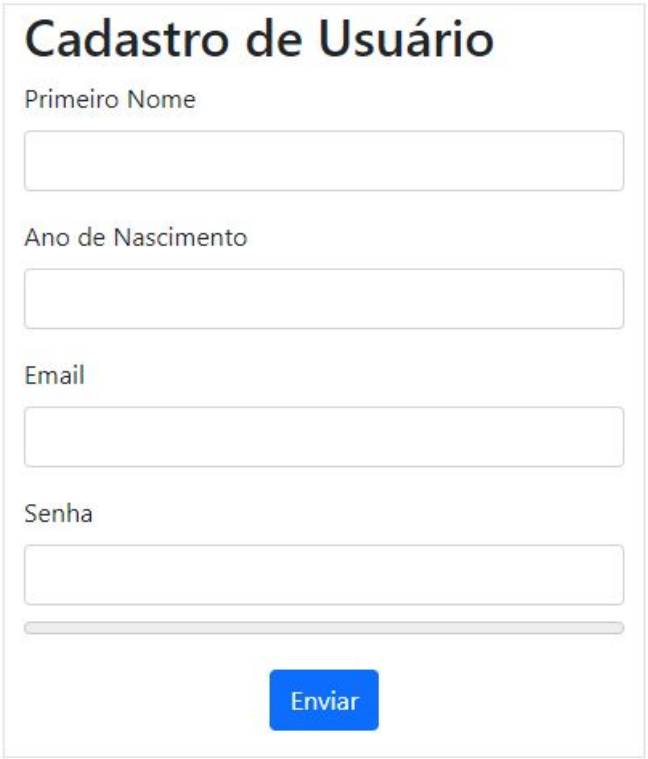



Suponha que você está participando de um projeto de desenvolvimento de um sistema web e precisa garantir a qualidade da implementação da seguinte história de usuário.

*HU: Como usuário gostaria de me cadastrar no sistema para ter acesso às funcionalidades e apoiar a realização das minhas atividades.*

Suponha também que o desenvolvimento da HU está sendo realizado por você. A partir da especificação de requisitos você precisa identificar os casos de teste para essa HU.

# Exemplo - HU: Cadastro de Usuário




Os requisitos para essa HU são os seguintes:

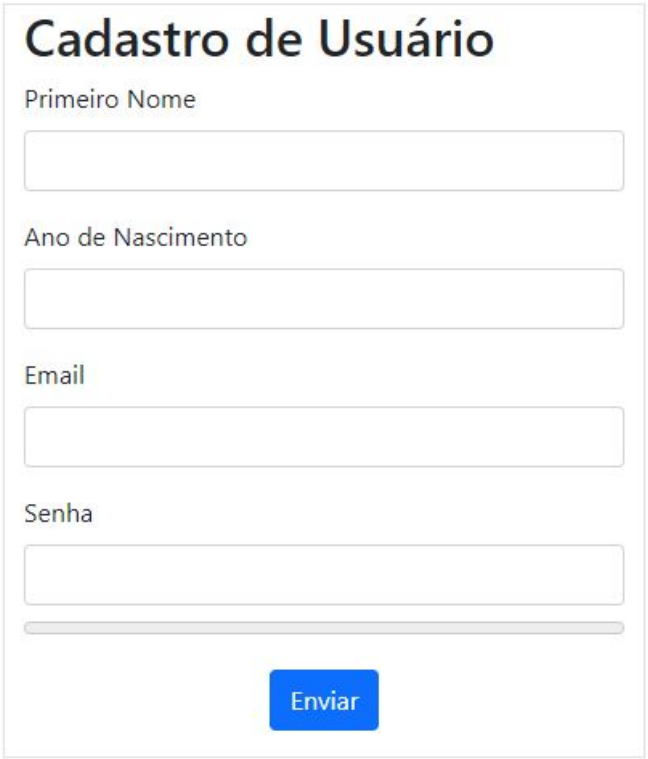
RF1: O sistema deve permitir o cadastro de um usuário considerando as informações:

- O nome do usuário, composto somente por letras e deve ser de comprimento maior ou igual a 6 e menor ou igual a 30

**Quais dados de entrada usariam para testar o script que valida o campo de nome de usuário, considerando os requisitos acima?**

**Qual o comportamento esperado para essas entradas?**

# Exemplo - HU: Cadastro de Usuário

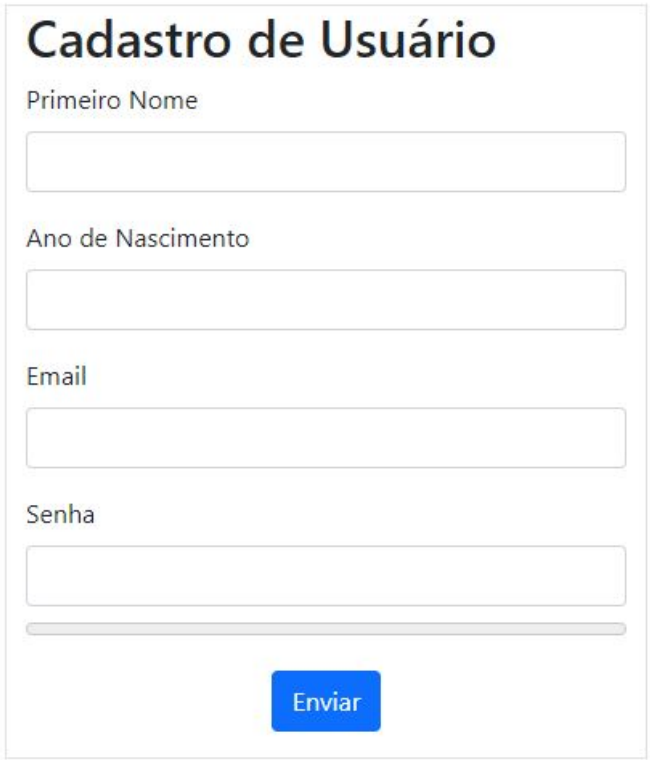



Os requisitos para essa HU são os seguintes:

RF1: O sistema deve permitir o cadastro de um usuário considerando as informações:

- O nome do usuário, composto somente por letras e deve ser de comprimento maior ou igual a 6 e menor ou igual a 30
- O ano de nascimento deve considerar pessoas nascidas nos últimos 120 anos
- o email, que deve conter letras e números seguido do '@' seguido de caracteres e/ou números seguido de '.' e finalizando em 'br, com, net, org'

# Exemplo - HU: Cadastro de Usuário




Os requisitos para essa HU são os seguintes:

**RF1.1:** Para que uma senha seja considerada válida, deve:

- ter entre 6 e 20 caracteres,
- ter pelo menos uma ocorrência de:
  - caractere especial, por exemplo: [@, #, %, &, !,+]
  - número
  - letra [a-z,A-Z]
- Não conter o nome ou o ano de nascimento do usuário

No caso da senha ser inválida, a função deve retornar a mensagem ***Senha inválida***.

No caso da senha ser válida, a função deve retornar o nível de segurança da senha: ***'fraca'***, ***'moderada'***, ou ***'forte'***, seguindo os seguintes requisitos:

- Uma senha **fraca** tem comprimento menor ou igual a 8 caracteres, contendo pelo menos um caractere especial e um número;
- Uma senha **moderada** tem mais de 8 caracteres, contendo pelo menos um caractere especial, um número e uma letra maiúscula;
- Uma senha **forte** tem mais de 12 caracteres, contendo mais de um caracter especial, mais de um número e mais de uma letra maiúscula.

# Qual o propósito da atividade de teste de software?

## O que produz falhas no software?



1 – Desenvolvedor comete um **engano** (*mistake*)



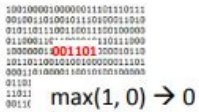

4 – Software **falha** (*failure*)



5 – Usuário sofre as consequências



2 – Software com **defeito** (*fault*)




3 – Defeito é exercitado e gera um **erro** (*error*)

## Como evitar falhas e garantir a qualidade do software (código)?

- **Depuração** - identificação da causa do erro e correção do erro
- **Verificação** - avaliar se o produto está sendo desenvolvido corretamente
- **Validação** - avaliar se o produto é o correto

**FUNDAMENTADAS NA ATIVIDADE DE TESTE DE SOFTWARE**

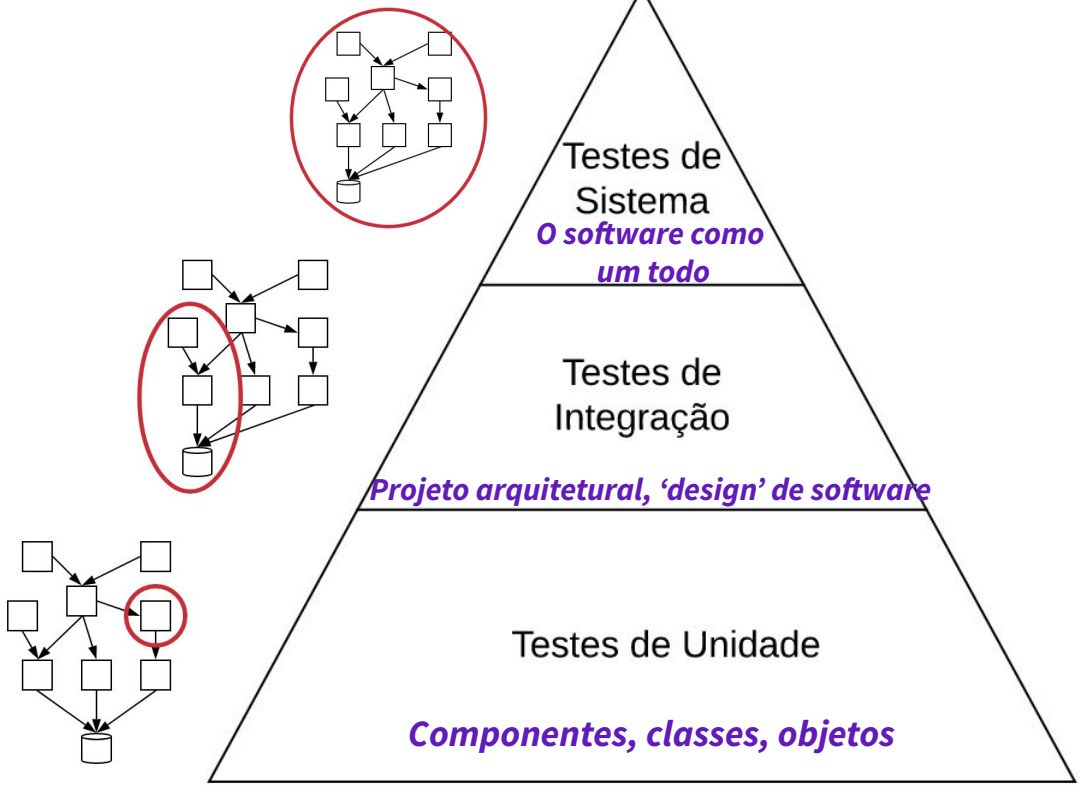
# *Fases de Teste*

## O que testamos?

**10% dos testes**

**20% dos testes**

**70% dos testes**

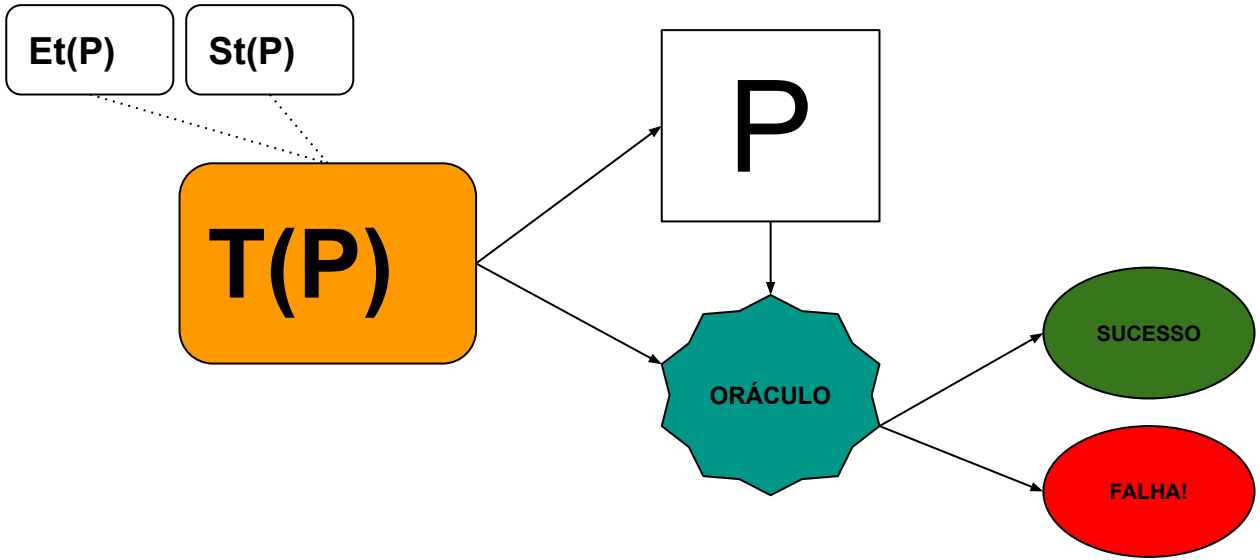



Maior granularidade
Menor quantidade
Mais lentos
Maior custo

P

Menor granularidade
Maior quantidade
Mais rápidos
Menor custo

# Cenário Típico da Atividade de Teste




**Et(P)** - Conjunto de entradas de teste do programa.
**St(P)** - Conjunto de saídas de teste do programa.
**T(P) - Conjunto de casos de teste do programa → Também denominados REQUISITOS DE TESTE**

# Caso de Teste

Par formado por $< ei, sj >$, onde:

$ei$ pertence a $Et(P)$

$sj$ pertence a $St(P)$

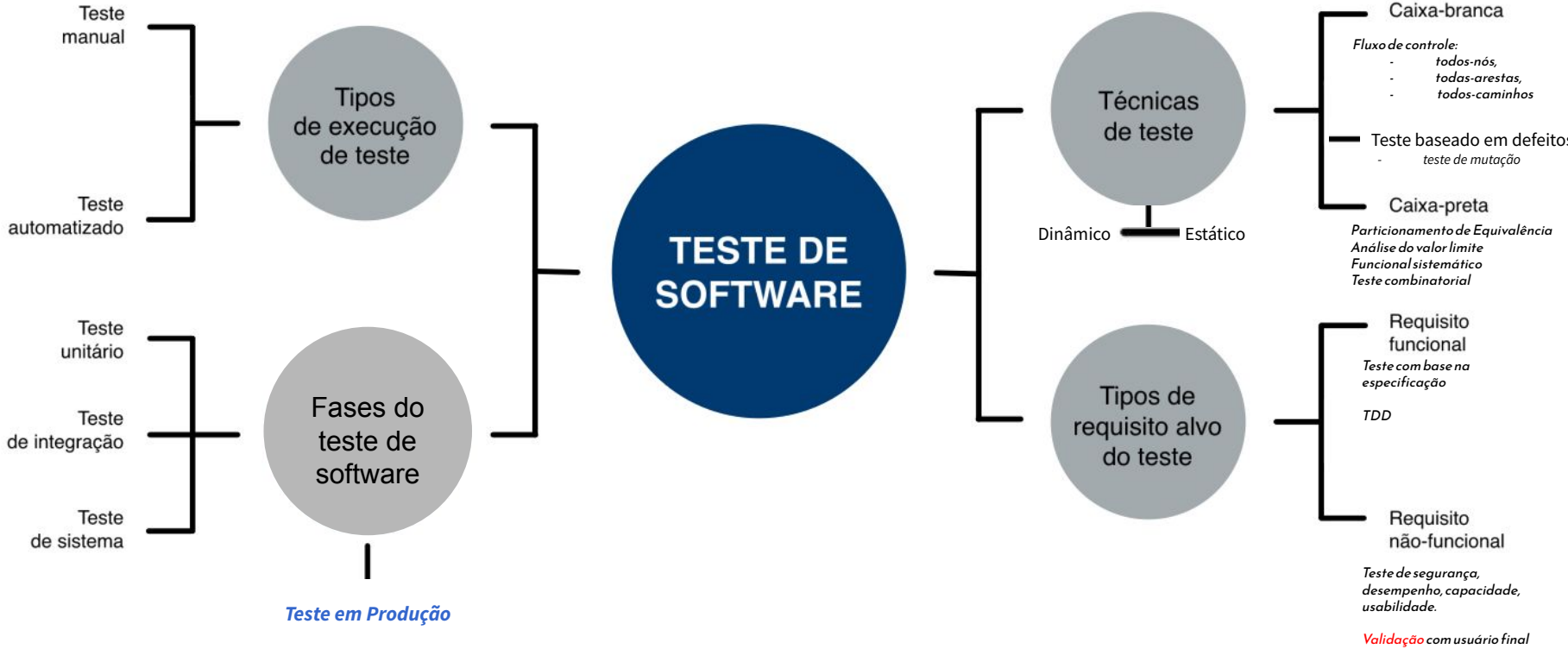
$sj$ é o resultado esperado para a execução do programa P com o dado de entrada de teste $ei$

# Técnicas de Seleção de Casos de Teste

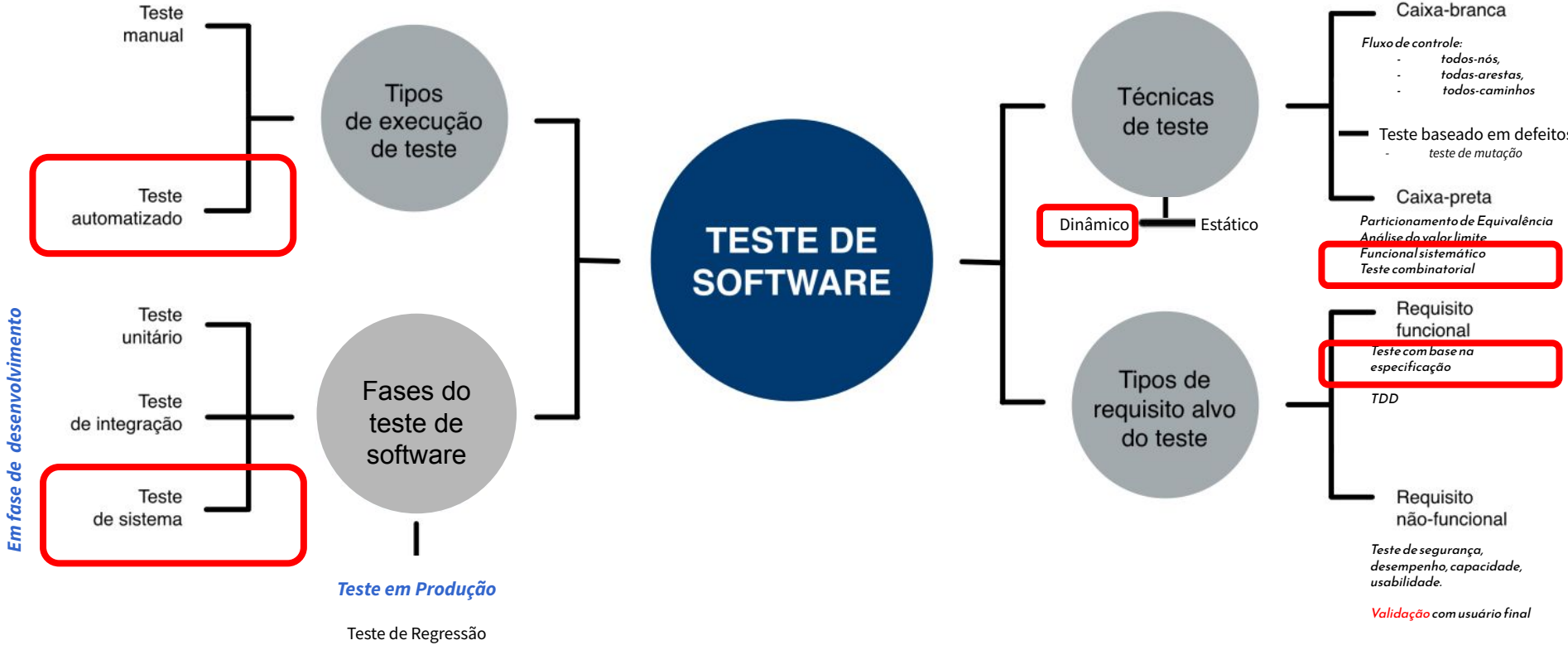
Dependendo do objetivo do testador, cada tipo de técnica propõe diferentes **critérios para identificar casos de teste** que permitam **achar o maior número de erros** no software.

1. **Teste Funcional**- Técnicas de ‘caixa preta’
   a. Particionamento de Equivalência
   b. Análise do valor limite
   **c. Funcional sistemático**
   **d. Teste combinatorial**
2. **Teste Estrutural** - Técnicas de ‘caixa branca’
   a. *Fluxo de controle:*
      i. todos-nós,
      ii. todas-arestas,
      iii. todos-caminhos
3. **Teste Não Funcional**

# Visão Geral da Atividade de Teste de Software



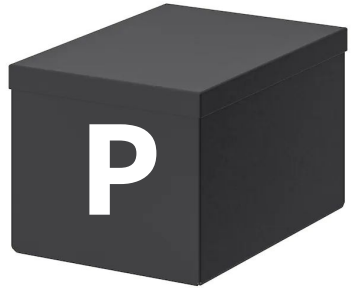
# Visão Geral da Atividade de Teste de Software



# Teste Funcional -- ‘Caixa preta’

**Objetivo**: Validar o software do ponto de vista do esperado pelo usuário

- Detalhes de implementação (código) não são considerados
- **Requer a especificação de requisitos/HU/UC** do produto de software para derivar os casos de teste
- **Não** se foca na implementação (código) do programa
- Pode ser **aplicada em TODAS as fases de teste** (unidade, integração, sistema)
- Pode ser **aplicada para testar qualquer P** (classe, método, componente, módulo, sistema …)



# Critério
# Teste Funcional (Caixa Preta)



## COMBINATORIAL

# Teste Funcional - Critério: Combinatorial

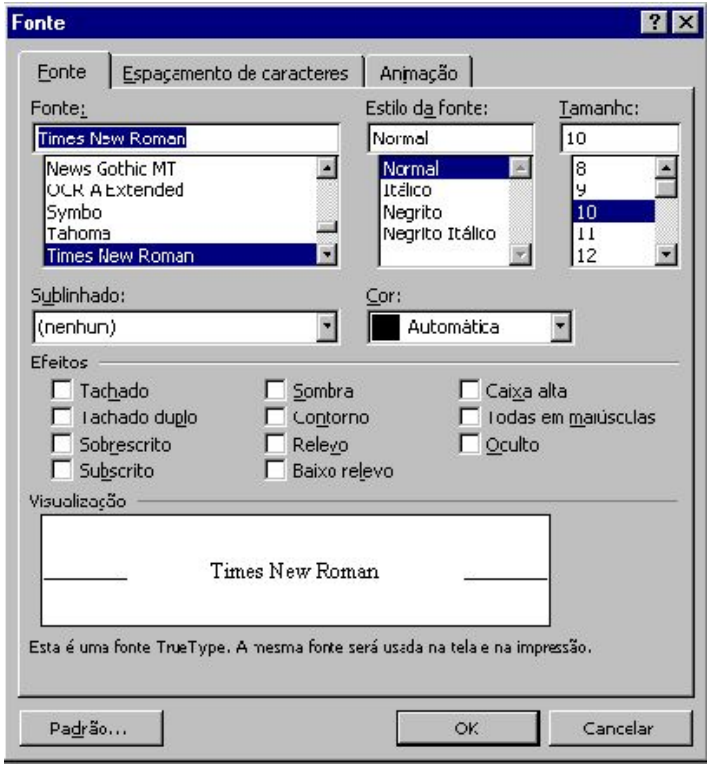
Usado para testar diferentes ***combinações de dados de entrada.***

Usa-se para testar interações entre métodos (***teste de integração***) através de GUI

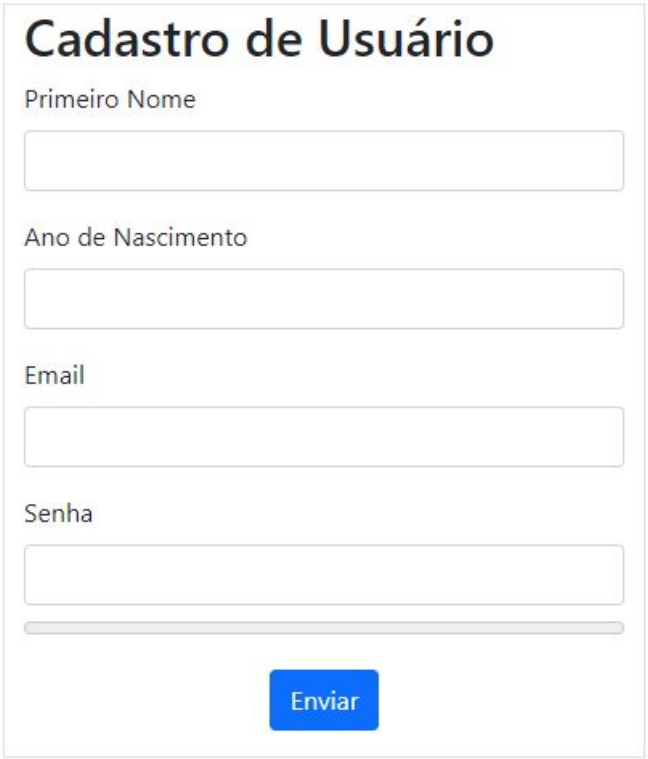
A **experiência/habilidades do testador** para selecionar os casos de teste é muito importante.

***Exemplo***: Para o programa de formatação de texto do Microsoft Word, podem chegar a existir 3 trilhões de possibilidades de entrada, considerando todas as combinações possíveis.

Só na opção de efeitos são $2^{11}$ = 2.048 combinações a testar.



# Exemplo - HU: Cadastro de Usuário

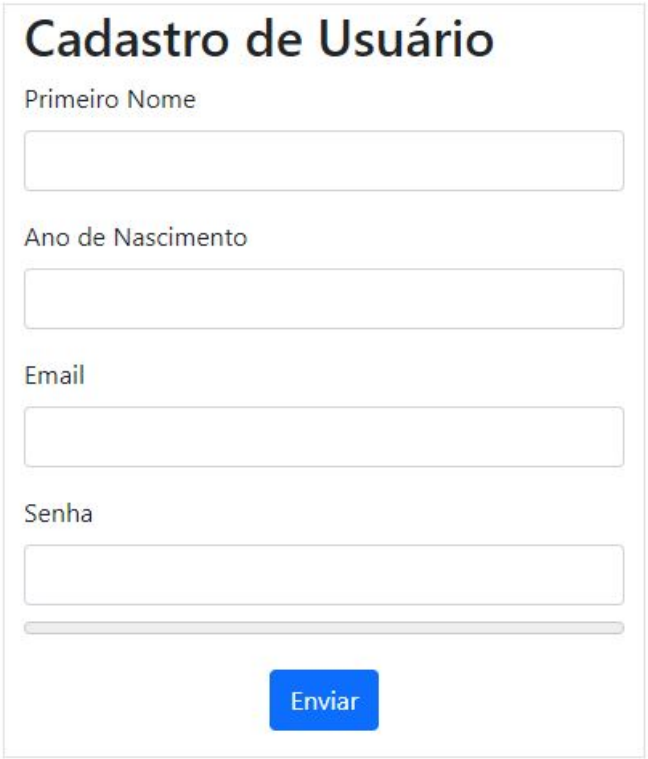



Suponha que você está participando de um projeto de desenvolvimento de um sistema web e como testador precisa garantir a qualidade da implementação da seguinte história de usuário.

*HU: Como usuário gostaria de me cadastrar no sistema para ter acesso às funcionalidades e apoiar a realização das minhas atividades.*

Suponha também que o desenvolvimento da HU está sendo realizado por você. A partir da especificação de requisitos você precisa identificar os casos de teste para essa HU.

# Exemplo - HU: Cadastro de Usuário




Os requisitos para essa HU são os seguintes:

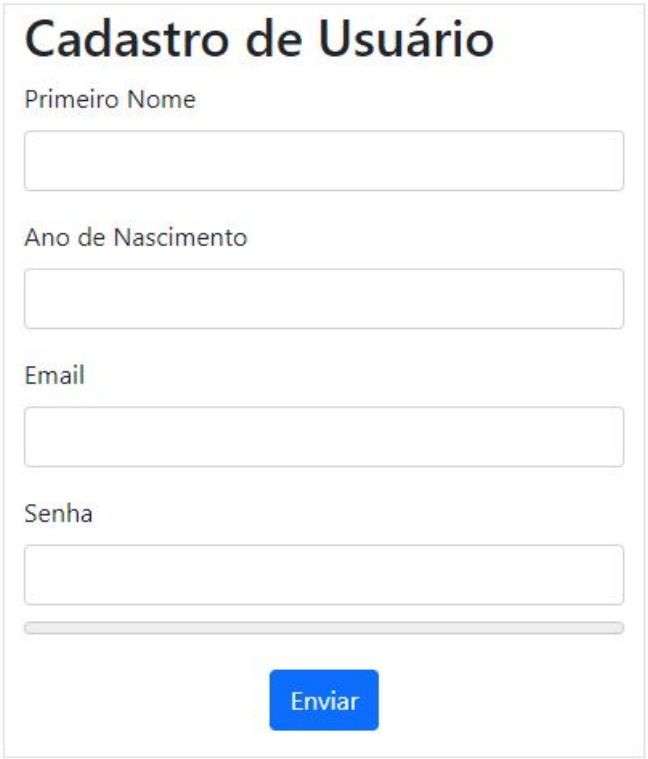
RF1: O sistema deve permitir o cadastro de um usuário considerando as informações:

- O nome do usuário, composto somente por letras e deve ser de comprimento maior ou igual a 6 e menor ou igual a 30

**Quais dados de entrada usariam para testar o script que valida o campo de nome de usuário, considerando os requisitos acima?**

**Qual o comportamento esperado para essas entradas?**

# Exemplo - HU: Cadastro de Usuário

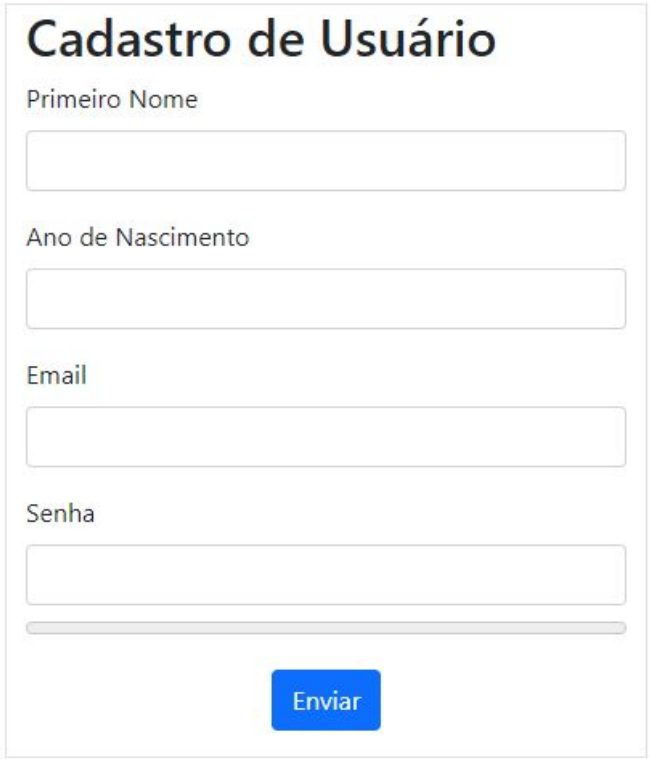



Os requisitos para essa HU são os seguintes:

RF1: O sistema deve permitir o cadastro de um usuário considerando as informações:

- O nome do usuário, composto somente por letras e deve ser de comprimento maior ou igual a 6 e menor ou igual a 30
- O ano de nascimento deve considerar pessoas nascidas nos últimos 120 anos
- o email, que deve conter letras e números seguido do '@' seguido de caracteres e/ou números seguido de '.' e finalizando em 'br, com, net, org'

# Exemplo - HU: Cadastro de Usuário




Os requisitos para essa HU são os seguintes:

**RF1.1:** Para que uma senha seja considerada válida, deve:

- ter entre 6 e 20 caracteres,
- ter pelo menos uma ocorrência de:
  - caractere especial, por exemplo: [@, #, %, &, !,+]
  - número
  - letra [a-z,A-Z]
- Não conter o nome ou o ano de nascimento do usuário

No caso da senha ser inválida, a função deve retornar a mensagem ***Senha inválida***.

No caso da senha ser válida, a função deve retornar o nível de segurança da senha: ***'fraca'***, ***'moderada'***, ou ***'forte'***, seguindo os seguintes requisitos:

- Uma senha **fraca** tem comprimento menor ou igual a 8 caracteres, contendo pelo menos um caractere especial e um número;
- Uma senha **moderada** tem mais de 8 caracteres, contendo pelo menos um caractere especial, um número e uma letra maiúscula;
- Uma senha **forte** tem mais de 12 caracteres, contendo mais de um caracter especial, mais de um número e mais de uma letra maiúscula.

## Exemplo usando o critério: Combinatorial




*Quantas combinações de entradas existem para o formulário de cadastro de usuário?*

# Exemplo usando o critério: Combinatorial




**COMBINAÇÕES DE ENTRADAS:**

- **Nome:** válido/inválido
- **Ano:** válido/inválido
- **Email:** válido/inválido
- **Senha:** inválida/fraca/moderada/forte

2X2X2X4 = 32 combinações de entradas

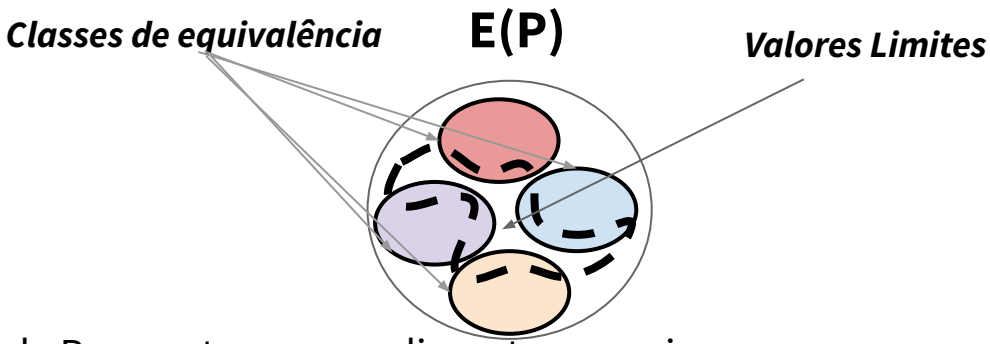
→ É necessário criar um conjunto T(P) de casos de teste que seja mínimo em tamanho, mas que consiga testar todas as possibilidades de entradas.

# Criando a Suíte de Testes T(P) usando Técnica Funcional  Combinatorial




1 - **Identificar as classes de equivalência para cada requisito a ser testado**, neste exemplo temos quatro requisitos para input de dados de usuário.

# Teste Funcional - Critério: Funcional Sistemático




Para cada requisito de P, executar o procedimento a seguir:

**Etapa 1:** Particionar os dados de entrada E(P) em **classes equivalentes *CEq***

**Etapa 2:** Seleção de ao menos dois dados de entrada de cada ***CEq*** e definição da saída esperada ***<ei,sj>***

- **2.1** Avaliação dos dados de entrada **no limite de cada partição**
- **2.2** Avaliação dos dados de entrada **subsequentes aos limites de cada partição**

# Etapa 1: Particionamento em Classes de Equivalência

Com base na ***especificação do produto de software*** (requisitos), seguir as seguintes **diretrizes** para definir as classes:

**1 -** Se **o requisito de dados de entrada de P,** especificar:

- (A) Um **intervalo** → *ex. valor entre [0, 10]*
  - definir uma classe válida para cada valor limite (0)(15) e duas inválidas para o valor de limite inferior (-1) e limite superior(11)
- (B) Uma **quantidade** → *ex. um número com dois dígitos (DD)*
  - definir uma classe válida (exatamente 2 números) e duas inválidas, para o limite inferior (somente um número) e limite superior (três números)
- (C) **Conjuntos de valores** determinados → ex. *Faixa etária entre: 0 - 15 (criança); 16 - 25 (jovem); 26 - 60 (adulto); >60 (idoso)*
  - definir uma classe válida (limite superior e inferior) para cada conjunto de valores, e uma duas classes inválidas: limite inferior *(-1)* e superior que contemple todos os conjuntos *(200)*
- (D) uma **restrição** ("deve ser assim") → ex. digite o CPF sem pontos nem traço
- definir uma classe válida (24577485921) e uma inválida (245.774.859-21)
- (E) Um conjunto ordenado → ex. {A,B,C,...,Z}
  - definir uma entrada de teste para cada valor limite (A)(Z)

**2 - Gerar o conjunto de casos de teste T(P)** selecionando um elemento de cada classe

- Para cada classe, escolhem-se de forma arbitrária os CASOS DE TESTE **<ei,sj>** de cada uma
- *Casos de teste são definidos para* ***classes com saídas válidas e inválidas***

**3 - Gerar o conjunto de casos de teste T(P)**

- Para cada entrada selecionada, define-se sua respectiva saída (válida, ou inválida) e especifica-se o caso de teste **<ei,sj>**
- *Casos de teste são definidos para* ***entradas válidas e inválidas***

# Criando a Suíte de Testes T(P) usando Técnica Funcional  Combinatorial




1 - **Identificar as classes de equivalência para cada requisito a ser testado**, neste exemplo temos quatro requisitos para input de dados de usuário.

2 - **Definir os casos de teste para as classes de equivalência**, usando os critérios funcionais para:

- *intervalo*
- *quantidade*
- *conjunto de valores*
- *restrição*
- *conjunto ordenado*

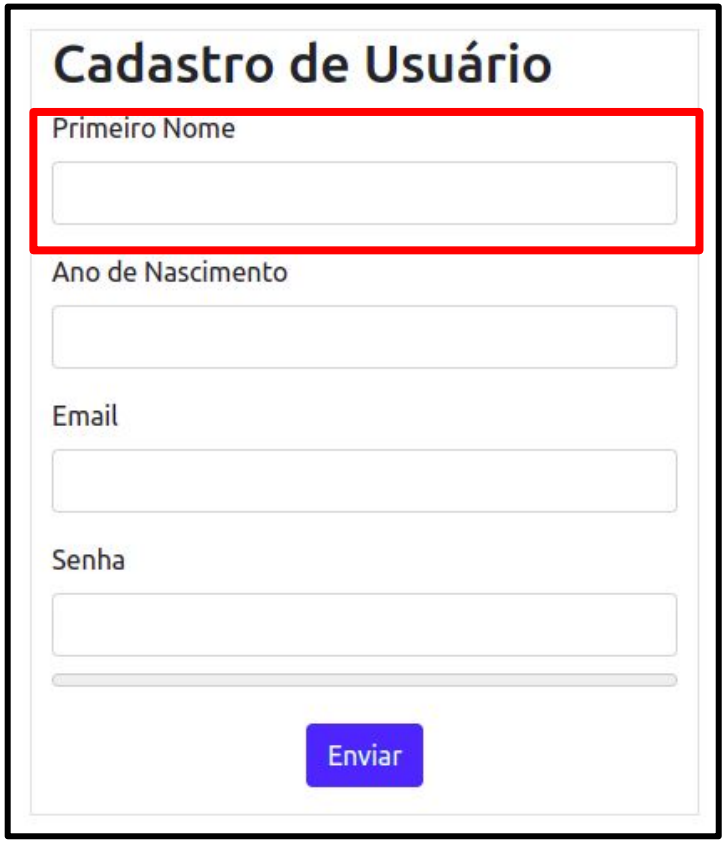
*Obs: verificar a necessidade de aplicar  o critério de valores limites para identificar os casos de teste.*

3 - **Documentar** os casos de teste em tabela

4 - **Implementar** os casos de teste em ferramenta para sua *automatização*

5 - Gerar **relatório de erros** encontrados

# 1 - Identificar as classes de equivalência para cada requisito a ser testado




***Requisito:*** *O nome do usuário, composto* ***(a) somente por letras*** *e deve ser de* ***(b) comprimento maior ou igual a 6 e menor ou igual a 30***

Critérios

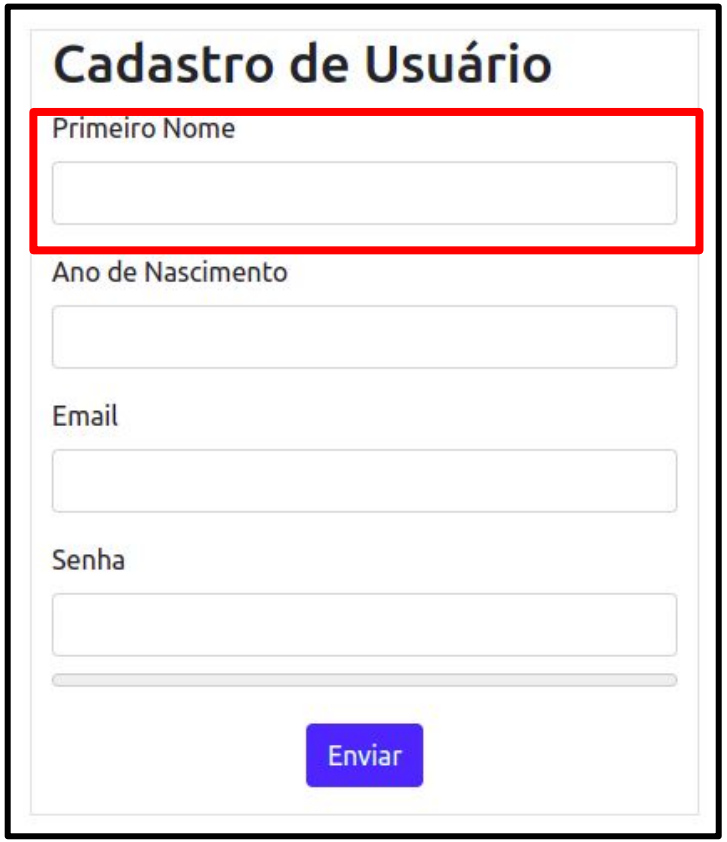
(a) Restrição: somente letras

(b) Intervalo: [6, 30]

Classes de equivalência

**Quantas classes de equivalência válidas e inválidas??**

# 1 - Identificar as classes de equivalência para cada requisito a ser testado




***Requisito:*** *O nome do usuário, composto* ***(a) somente por letras*** *e deve ser de* ***(b) comprimento maior ou igual a 6***

Critérios

(a) Restrição: somente letras

(b) Intervalo: [6, 30]

Classes de equivalência

(a) **CEq1:** nome somente com letras - **saída**: nome válido

**CEq2:** nome com letras, caracteres e números - **saída**: nome inválido

(b) **CEq3**: nome com comprimento >= 6 - **saída**: nome válido

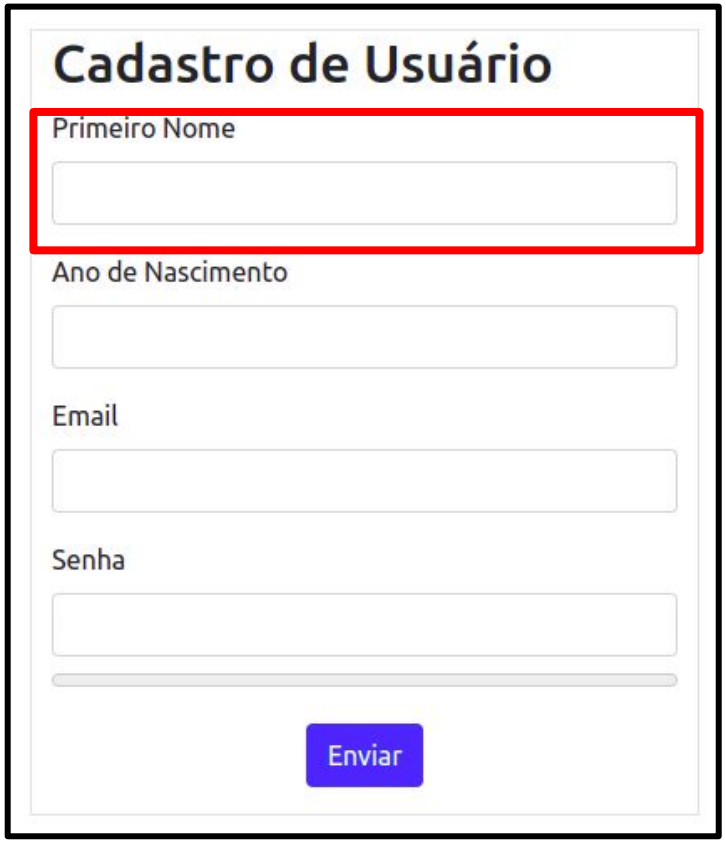
**CEq4**: nome com comprimento <= 30 - **saída**: nome válido

**CEq5**: nome com comprimento < 6 - **saída**: nome inválido

**CEq6** = nome com comprimento > 30 - **saída**: nome inválido

**CEq7**: nome vazio - **saída**: nome inválido

# 2 - Análise dos Valores limites em classes de equivalência CEq




***Requisito:*** *O nome do usuário, composto* ***(a) somente por letras*** *e deve ser de* ***(b) comprimento maior ou igual a 6***

Critérios

(a) Restrição: somente letras

(b) Intervalo: [6, 30]

Classes de equivalência

(a) **CEq1:** nome somente com letras - **saída**: nome válido

**CEq2:** nome com letras, caracteres e números - **saída**: nome inválido

(b) **CEq3**: nome com comprimento 6 - **saída**: nome válido

**CEq4**: nome com comprimento 30 - **saída**: nome válido

**CEq5**: nome com comprimento 5 - **saída**: nome inválido

**CEq6**: nome com comprimento 31 - **saída**: nome inválido

**CEq7**: nome vazio - **saída**: nome inválido

***Aplicação dos critérios de valores limites no intervalo [6,30]*

# 3 - Definir os casos de teste para as classes de equivalência

***Requisito:*** *O nome do usuário, composto* ***(a) somente por letras*** *e deve ser de* ***(b) cumprimento maior a 6***

Critérios

(a) Restrição: somente letras

(b) Intervalo: [6, 30]

Classes de equivalência

(a) **CEq1:** nome somente com letras - **saída**: nome válido

**CEq2:** nome com letras, caracteres e números - **saída**: nome inválido

(b) **CEq3**: nome com comprimento 6 - **saída**: nome válido

**CEq4**: nome com comprimento 30 - **saída**: nome válido

**CEq5**: nome com comprimento 5 - **saída**: nome inválido

**CEq6**: nome com comprimento 31 - **saída**: nome inválido

**CEq7**: nome vazio - **saída**: nome inválido

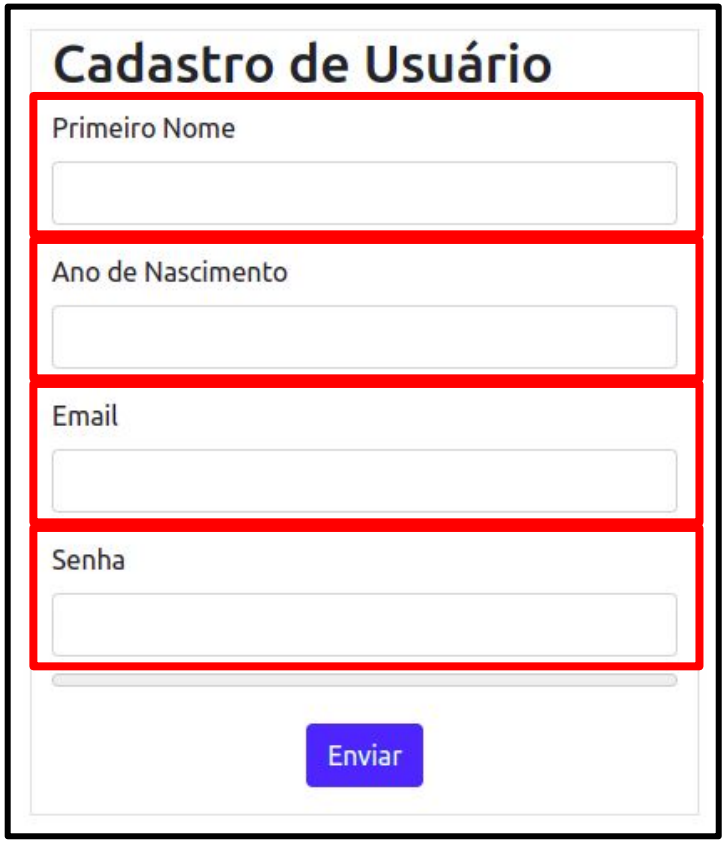
***Aplicação dos critérios de valores limites no intervalo [6,30]*

Casos de Teste <entrada, saída>

- caso: < 'Julian', nome válido> **(CEq1, CEq3)**
- caso: < 'M@r1a', nome inválido> **(CEq2, CEq5)**
- caso: < 'Maria Aparecida Lucinda Ferreiras', nome válido> **(CEq4)**
- caso: < 'Maria Aparecida Lucinda de Ferreira', nome inválido> **(CEq6)**
- caso: < '', nome inválido> **(CEq7)**

**Suíte de Teste T(P) =** { < 'Julian', nome válido> **;** < 'M@r1a', nome inválido> **;** < 'Maria Aparecida Lucinda Ferreiras', nome válido> **;** < 'Maria Aparecida Lucinda de Ferreira', nome inválido> **;** < '', nome inválido> }

# Criando a Suíte de Testes T(P) usando Técnica Funcional  Combinatorial




1 - **Identificar as classes de equivalência para cada requisito a ser testado**, neste exemplo temos quatro requisitos para input de dados de usuário.

2 - **Definir os casos de teste para as classes de equivalência**, usando os critérios funcionais para:

- *intervalo*
- *quantidade*
- *conjunto de valores*
- *restrição*
- *conjunto ordenado*

*Obs: verificar a necessidade de aplicar  o critério de valores limites para identificar os casos de teste.*

3 - **Documentar** os casos de teste em tabela

4 - **Implementar** os casos de teste em ferramenta para sua *automatização*

# 3 - Documentando os casos de teste funcional combinatorial




| ID Caso de Teste | Combinação | Caso de Teste (representante) |
|---|---|---|
| CT1 | **entrada**:<br>[nome: válido,<br>ano: válido,<br>email: válido,<br>senha: válida]<br><br>**saída**: cadastro válido | **e1:** [ nome: 'Julian', ano: 1991, email: 'jul@usp.br', senha: '#@49%No74']<br>**s1:** Cadastro Válido<br><br>**e2:** [ nome: 'Maria Aparecida Lucinda Ferreiras', ano: 1991, email: 'malf@usp.br', senha: '#@49%No74' ]<br>**s2:** Cadastro Válido |
| CT2 | **entrada**:<br>[nome: inválido,<br>ano: válido,<br>email: válido,<br>senha: válida]<br><br>**saída**: cadastro inválido | **e3:** [nome: 'M@r1a', ano: 1991, email: 'maria@usp.br', senha: ''#@49%No74']<br>**s3:** Cadastro inválido<br><br>**e4:** [nome: 'MariaAparecidaLucindadeFerreira', ano: 1991, email: 'maria@usp.br', senha: ''#@49%No74']<br>**s4:** Cadastro inválido<br><br>**e5:** [nome: ' ', ano: 1991, email: 'maria@usp.br', senha: ''#@49%No74']<br>**s5:** Cadastro inválido |

# 3 - Documentando os casos de teste funcional combinatorial




| ID Caso de Teste | Combinação | Caso de Teste (representante) |
|---|---|---|
| CT3 | **entrada**: [ nome: válido, ano: inválido, email:válido, senha: válido ]<br>**saída**: cadastro inválido | |
| CT4 | **entrada**: [ nome: válido, ano: válido, email: inválido, senha: válido ]<br>**saída**: cadastro inválido | |
| CT5 | **entrada**: nome: válido, ano: válido, email: válido, senha: inválido<br>**saída**: cadastro inválido | |
| | **… continuar as combinações …** | |
| CT32 | **entrada**: [ nome: inválido, ano: inválido, email: inválido, senha: inválida ]<br>**saída**: cadastro inválido | **ex:** [nome: 'M@r1a', ano: 2025, email: 'mariausp.br', senha: ''123456']<br>**sx:** Cadastro inválido |

# 4 - Implementando os Casos Testes Usando Ferramentas de Automação

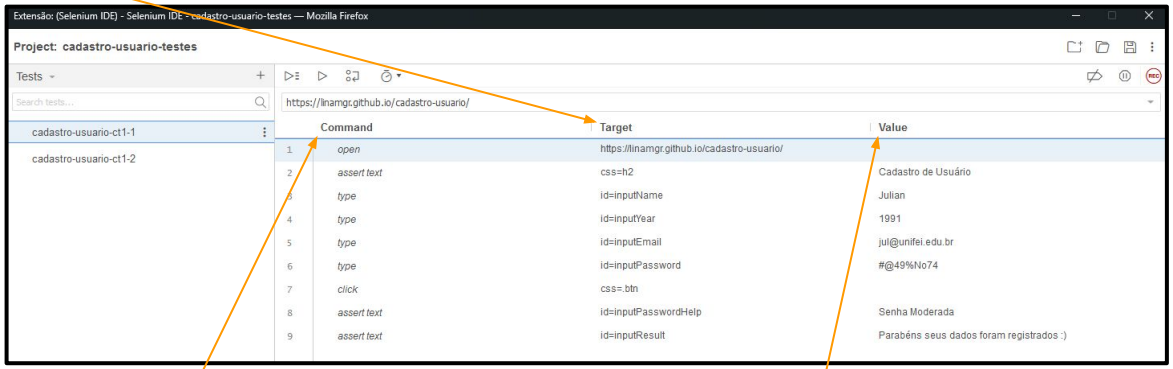
## Exemplo: Automação de Testes de Interface (E2E) com Selenium



# 4 - Implementando os Testes - Automação de Testes de Interface (E2E) com Selenium

**TARGET =** Elemento html (caixa de texto, label, botão) que estamos considerando no caso de teste



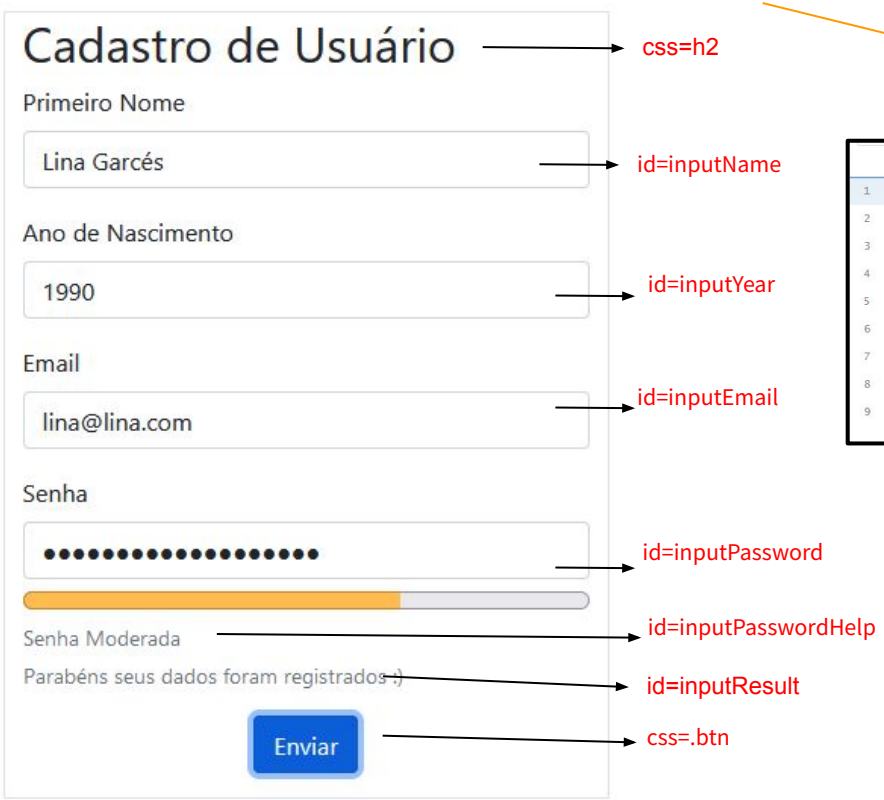

**Comandos do Selenium**
open, type (digitar), click, assert text (comparar texto)

**VALUE =** valor para o objeto que o testador deseja inserir (usando os comandos type, click, select) como entrada OU obter como resposta (usando o comando assert)

Lista dos comandos: https://www.selenium.dev/selenium-ide/docs/en/api/commands

# Automação de Testes de Interface (E2E) com Selenium

**TARGET = Elemento html que estamos considerando no caso de teste**




css=h2

id=inputName

id=inputYear

id=inputEmail

id=inputPassword

id=inputPasswordHelp

id=inputResult

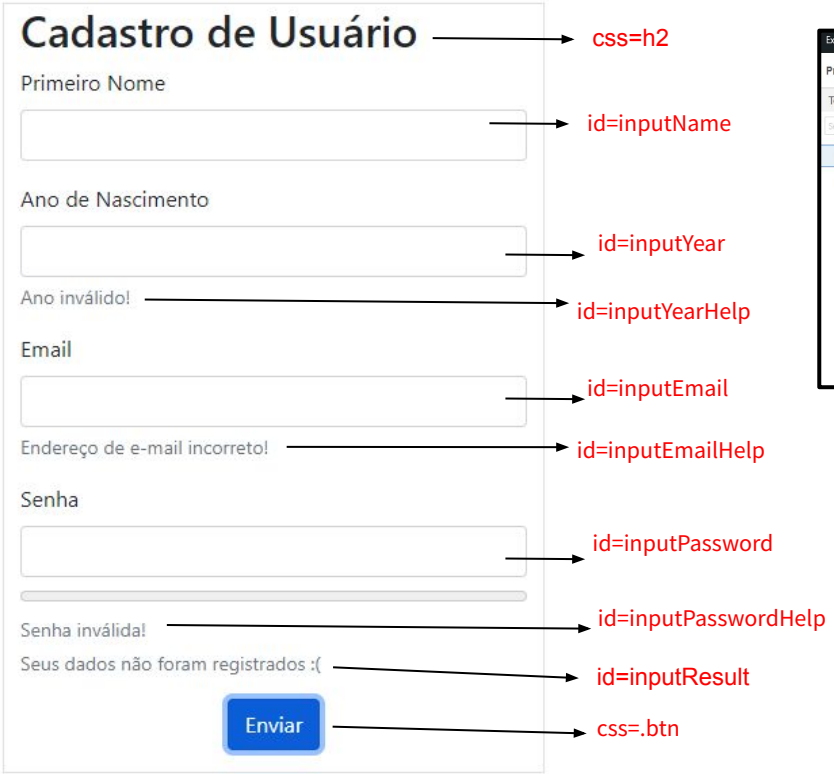
css=.btn

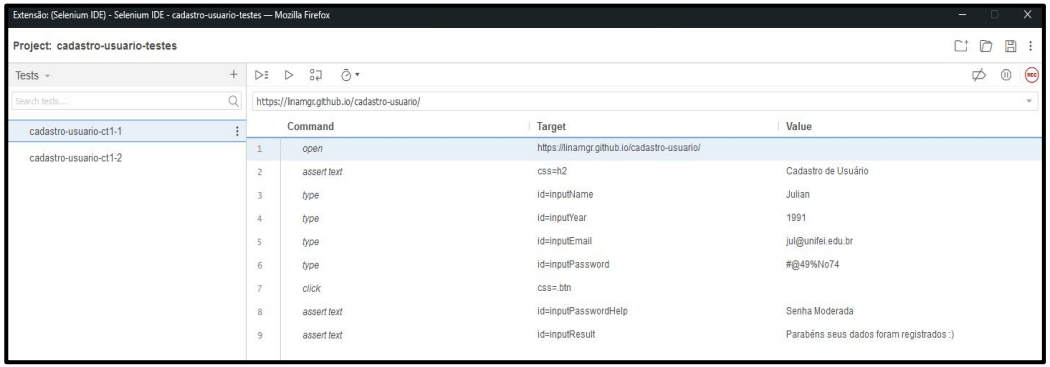
| | Command | Target | Value |
|---|---|---|---|
| 1 | open | https://linamgr.github.io/cadastro-usuario/ | |
| 2 | assert text | css=h2 | Cadastro de Usuário |
| 3 | type | id=inputName | Julian |
| 4 | type | id=inputYear | 1991 |
| 5 | type | id=inputEmail | jul@unifei.edu.br |
| 6 | type | id=inputPassword | #@49%No74 |
| 7 | click | css=.btn | |
| 8 | assert text | id=inputPasswordHelp | Senha Moderada |
| 9 | assert text | id=inputResult | Parabéns seus dados foram registrados :) |

**VALUE = valor para o objeto que o testador deseja inserir como entrada OU obter como resposta**

# Automação de Testes de Interface (E2E) com Selenium

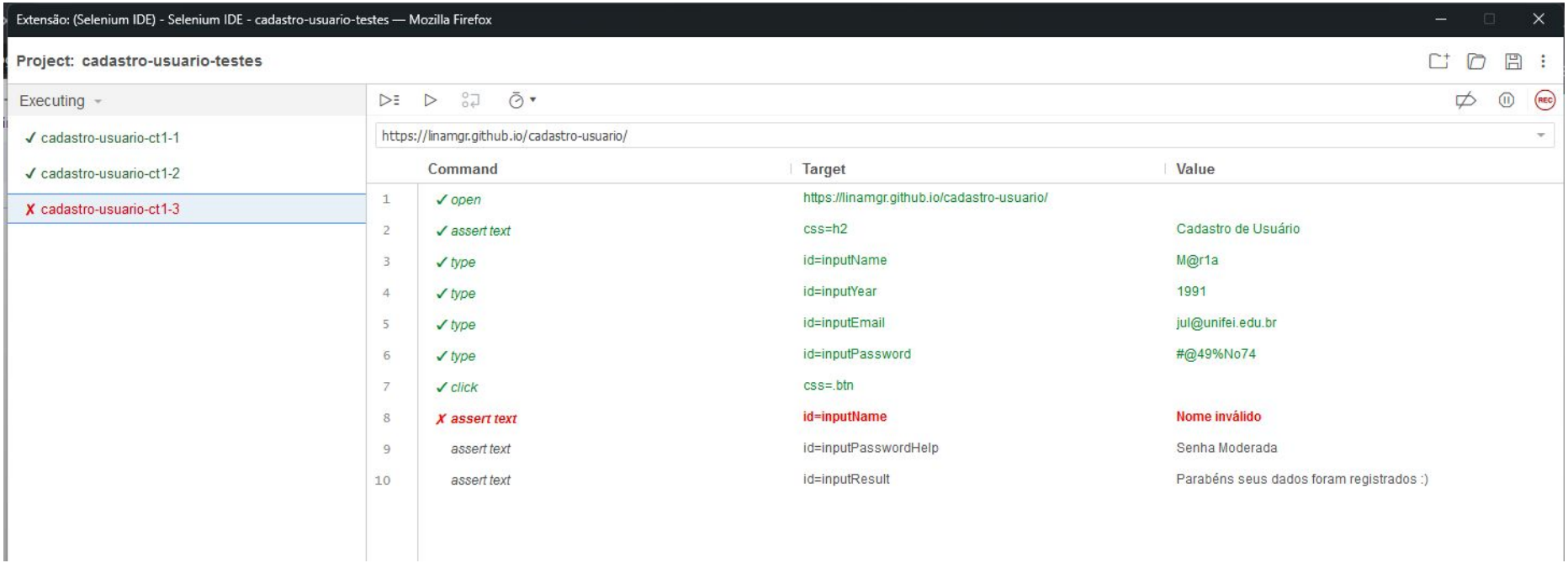
**TARGET = Elemento HTML que estamos considerando no caso de teste**




**VALUE = valor para o objeto que o testador deseja inserir como entrada OU obter como resposta**

# Resultados da Execução da suíte de testes com Selenium



# Referências

Márcio E. Delamaro, José C. Maldonado, Mario Jino. Introdução ao Teste de Software. 2a Ed. SBC, Elsevier. 2016.

Capítulo 17 do Livro: Rogers Pressman. Engenharia de Software. Uma Abordagem Profissional. 8a. Ed. 2016.

Capítulo 8 do Livro: Marco Tulio Valente. Engenharia de Software Moderna. Princípios e práticas para desenvolvimento de software com produtividade. 2019. Disponível online: https://engsoftmoderna.info/cap8.html

Curso online: Delamaro, M., Barbosa, E., Fassbinder, A., Andrade, S., Souza, D. Introdução ao Teste de Software. https://www.coursera.org/learn/intro-teste-de-software#syllabus